

Computer Organisation

Topic 12

Memory Management and Virtual Memory

Outline

- Memory Management
 - Swapping
 - Partitioning
 - Paging
- Virtual Memory
 - Demand paging
 - Page table structure
 - Translation Lookaside Buffer
 - Segmentation
- Pentium II/PowerPC Memory Management

Memory Management

- Uni-program
 - Memory split into two
 - One for Operating System (monitor)
 - One for currently executing program
- Multi-program
 - “User” part is sub-divided and shared among active processes

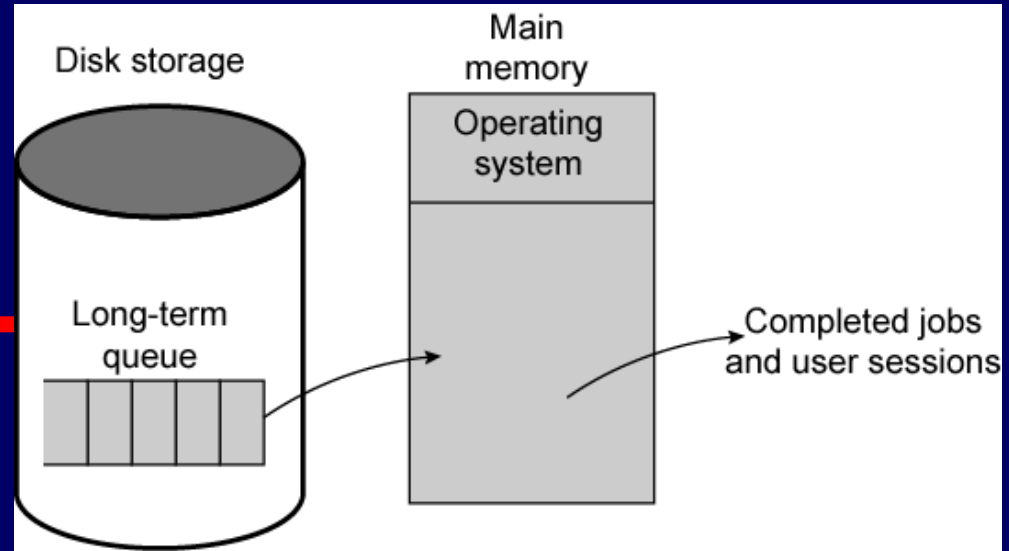
Swapping

- Problem: I/O is so slow compared with CPU that even in multi-programming system, CPU can be idle most of the time
- Solutions:
 - Increase main memory
 - Expensive
 - Leads to larger programs
 - Swapping

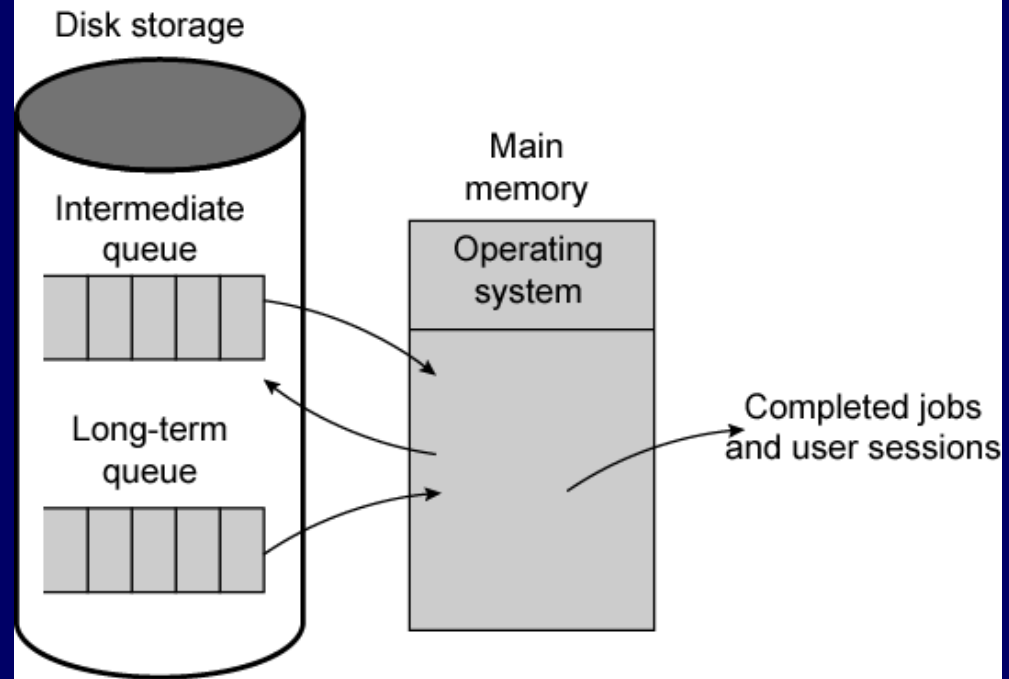
What is Swapping?

- Long term queue of processes stored on disk
- Processes “swapped” in as space becomes available
- As a process completes it is moved out of main memory
- If none of the processes in memory are ready (i.e. all I/O blocked)
 - Swap out a blocked process to intermediate queue
 - Swap in a ready process or a new process
 - But swapping is an I/O process...

Use of Swapping



(a) Simple job scheduling

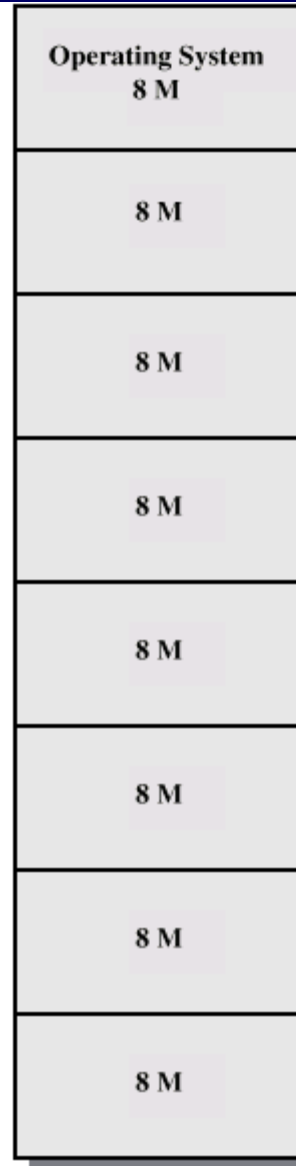


(b) Swapping

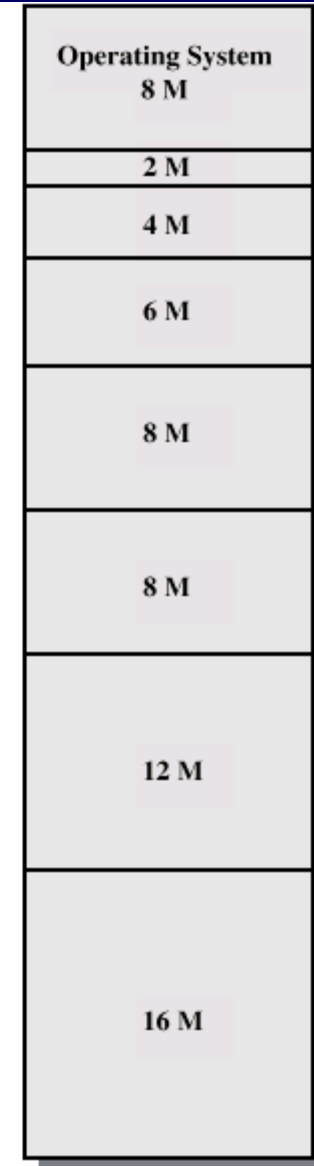
Partitioning

- Splitting memory into sections to allocate to processes (including Operating System)
- Fixed-sized partitions
 - May not be equal size
 - Process is fitted into smallest hole that will take it (best fit)
 - Some wasted memory
 - Leads to variable sized partitions

Fixed Partitioning



(a) Equal-size partitions



(b) Unequal-size partitions

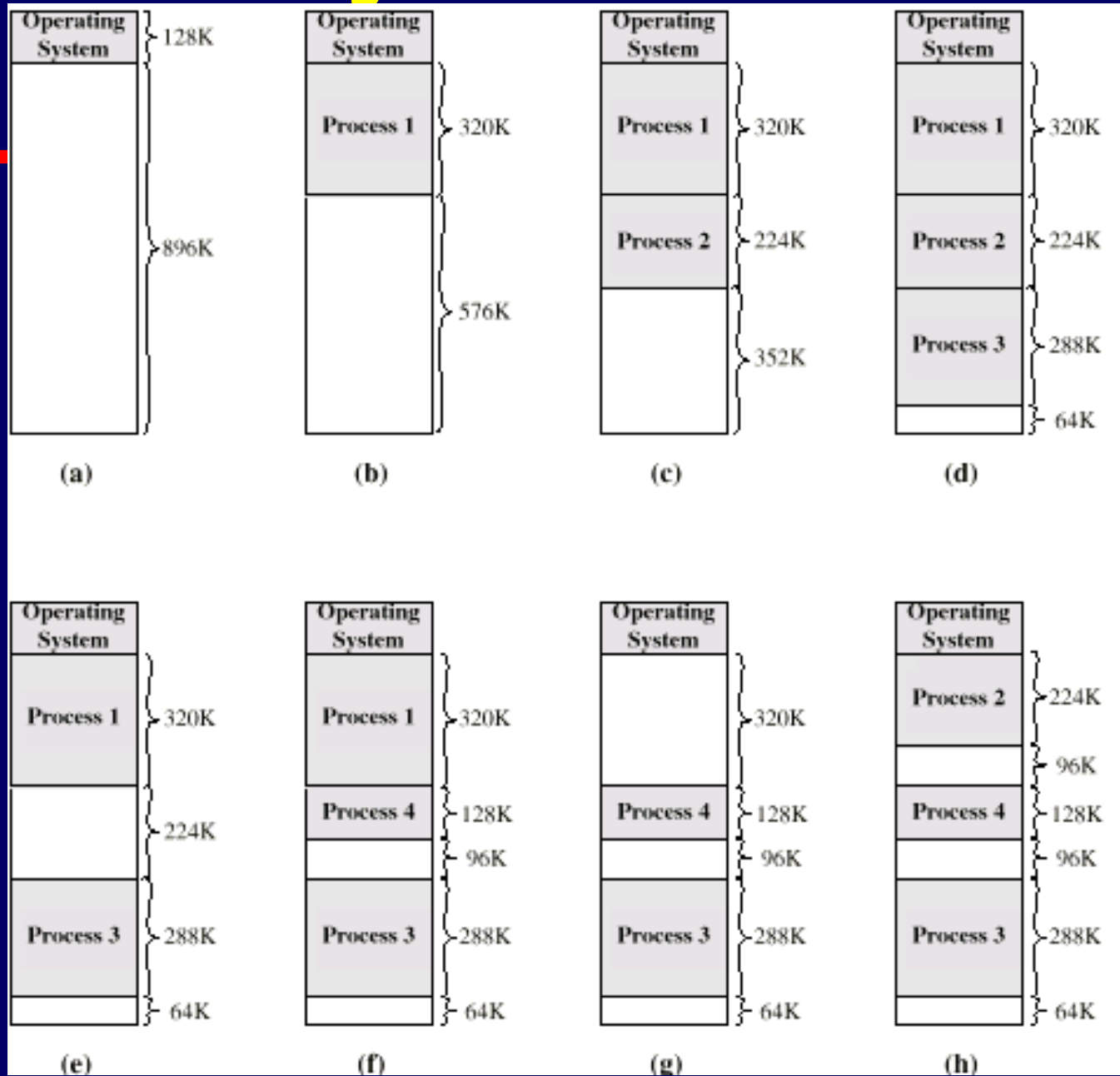
Variable Sized Partitions (1)

- Allocate exactly the required memory to a process
- This leads to a hole at the end of memory, too small to use
 - Only one small hole - less waste
- When all processes are blocked, swap out a process and bring in another
- New process may be smaller than swapped out process
- Another hole

Variable Sized Partitions (2)

- Eventually have lots of holes (fragmentation)
- Solutions:
 - Coalesce - Join adjacent holes into one large hole
 - Compaction - From time to time go through memory and move all hole into one free block (c.f. disk de-fragmentation)

Effect of Dynamic Partitioning



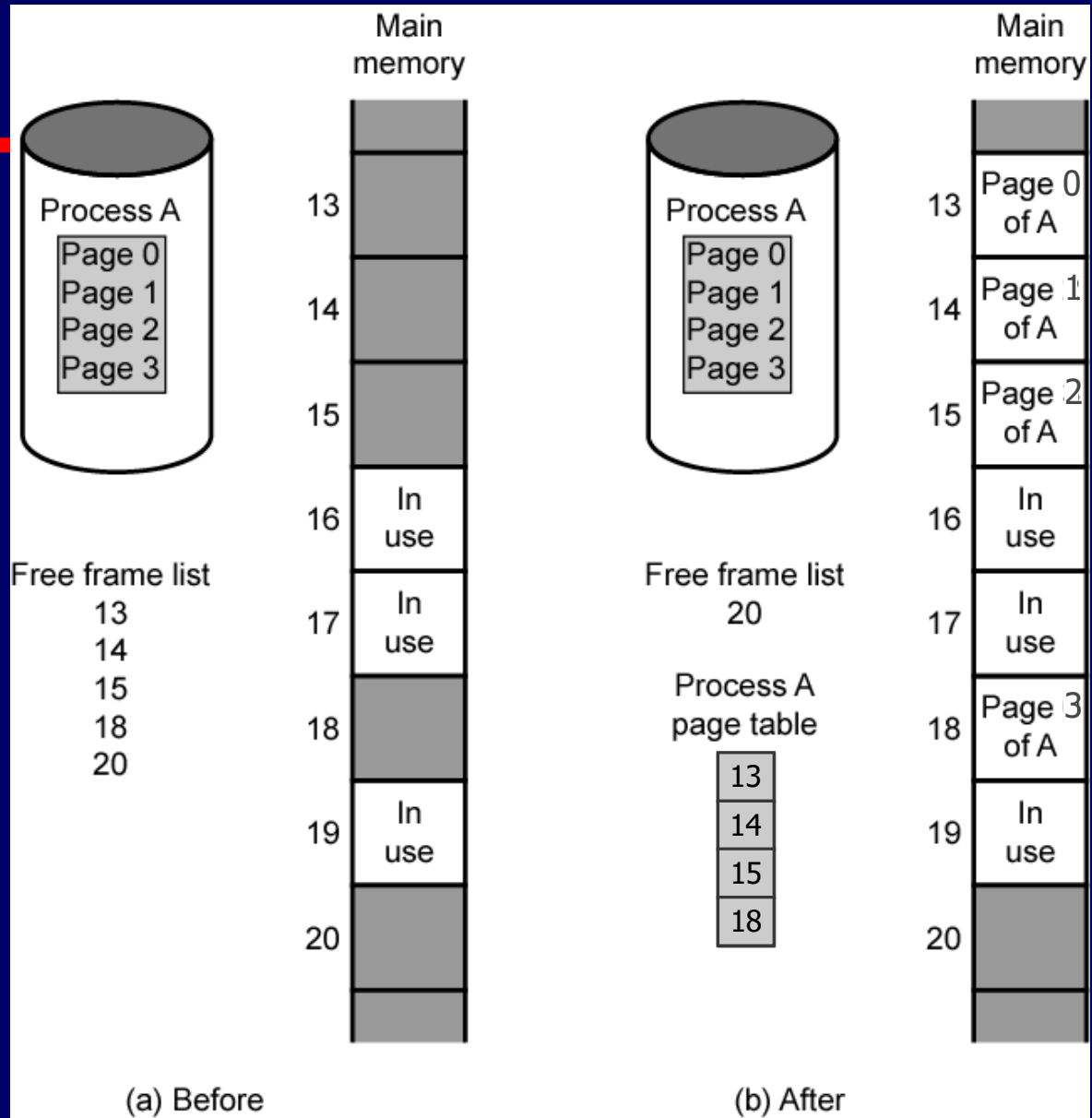
Relocation

- Instructions contain addresses
 - Locations of data
 - Addresses for instructions (branching)
- No guarantee that process will load into the same place in memory after swapping back in
 - To solve this problem, distinction is made between logical and physical addresses
 - Logical address: relative to beginning of program
 - Physical address: actual location in memory (this time)
 - Automatic conversion using base address

Paging

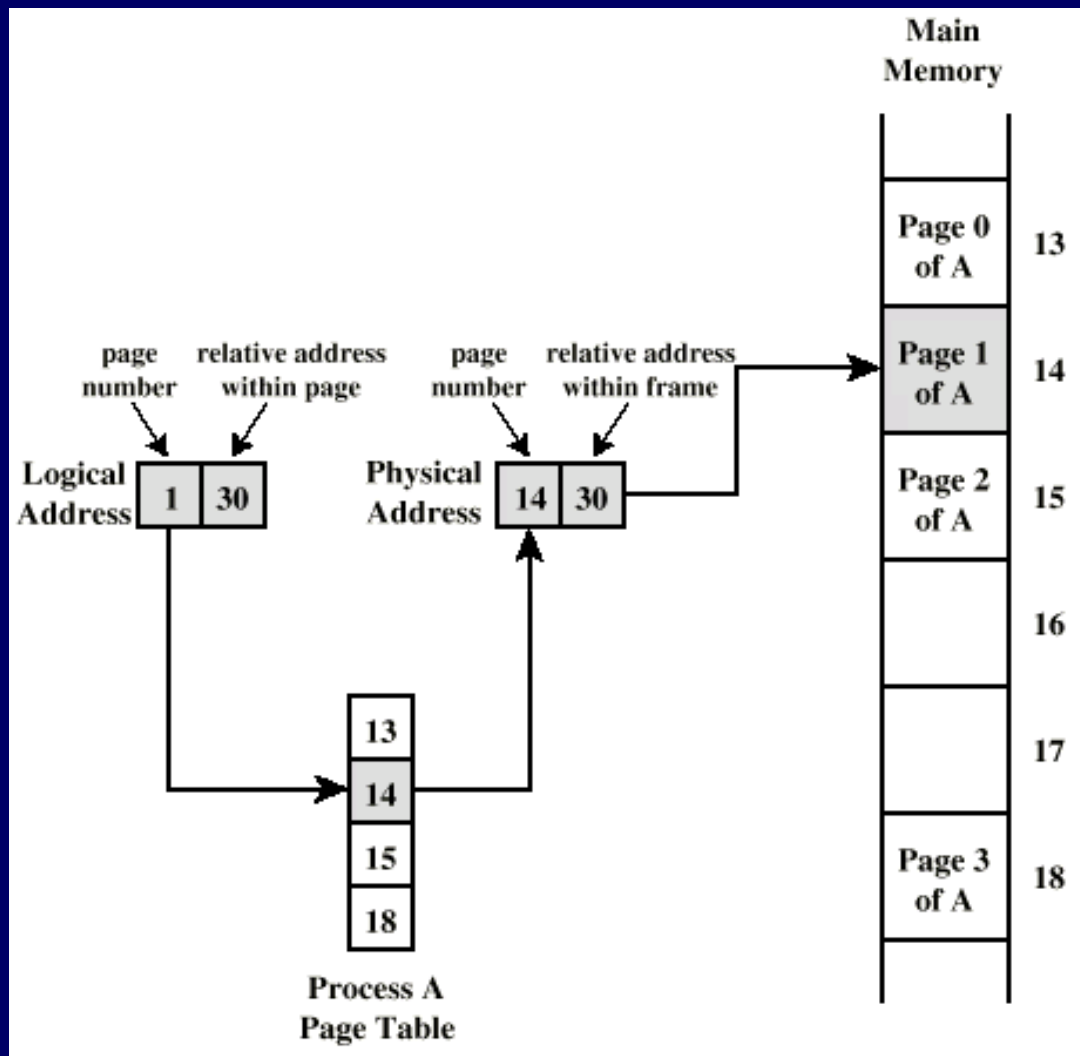
- Split memory into equal sized, small chunks: page frames
- Split programs (processes) into equal sized small chunks: pages
- Allocate the required number page frames to a process
- Operating System maintains list of free frames
- A process does not require contiguous page frames
- Use page table to keep track

Allocation of Free Frames



Paging

Logical and Physical Addresses



Outline

- Memory Management
 - Swapping
 - Partitioning
 - Paging
- Virtual Memory
 - Demand paging
 - Page table structure
 - Translation Lookaside Buffer
 - Segmentation
- Pentium II/PowerPC Memory Management

Virtual Memory

- Demand paging
 - Do not require all pages of a process in memory
 - Bring in pages as required
 - If page not in main memory (branch or requires data), page fault is triggered
- Page fault
 - Required page is not in memory
 - Operating System must swap in required page
 - May need to swap out a page to make space
 - Select page to throw out based on recent history
 - If page thrown out just before it is used, it must be immediately brought in again
 - Too many of this can lead to “trashing”

Thrashing

- Too many processes in too little memory
- Operating System spends all its time swapping
- Little or no real work is done
- Disk light is on all the time

- Solutions
 - Good page replacement algorithms
 - Reduce number of processes running
 - Fit more memory

Bonus

- We do not need all of a process in memory for it to run
- We can swap in pages as required
- So, we can now run processes that are bigger than total memory available
- Main memory is called real memory
- User/programmer sees much bigger memory - virtual memory

Page Table Structure

- Basic mechanism
 - Translation of a virtual/logical address (page number and offset) into a physical address (frame number and offset)
 - User page table
 - Register holds starting address of page table for a process
 - Page number of virtual address is used to index page table and look up corresponding frame number
 - This is combined with offset portion of virtual address to produce real address

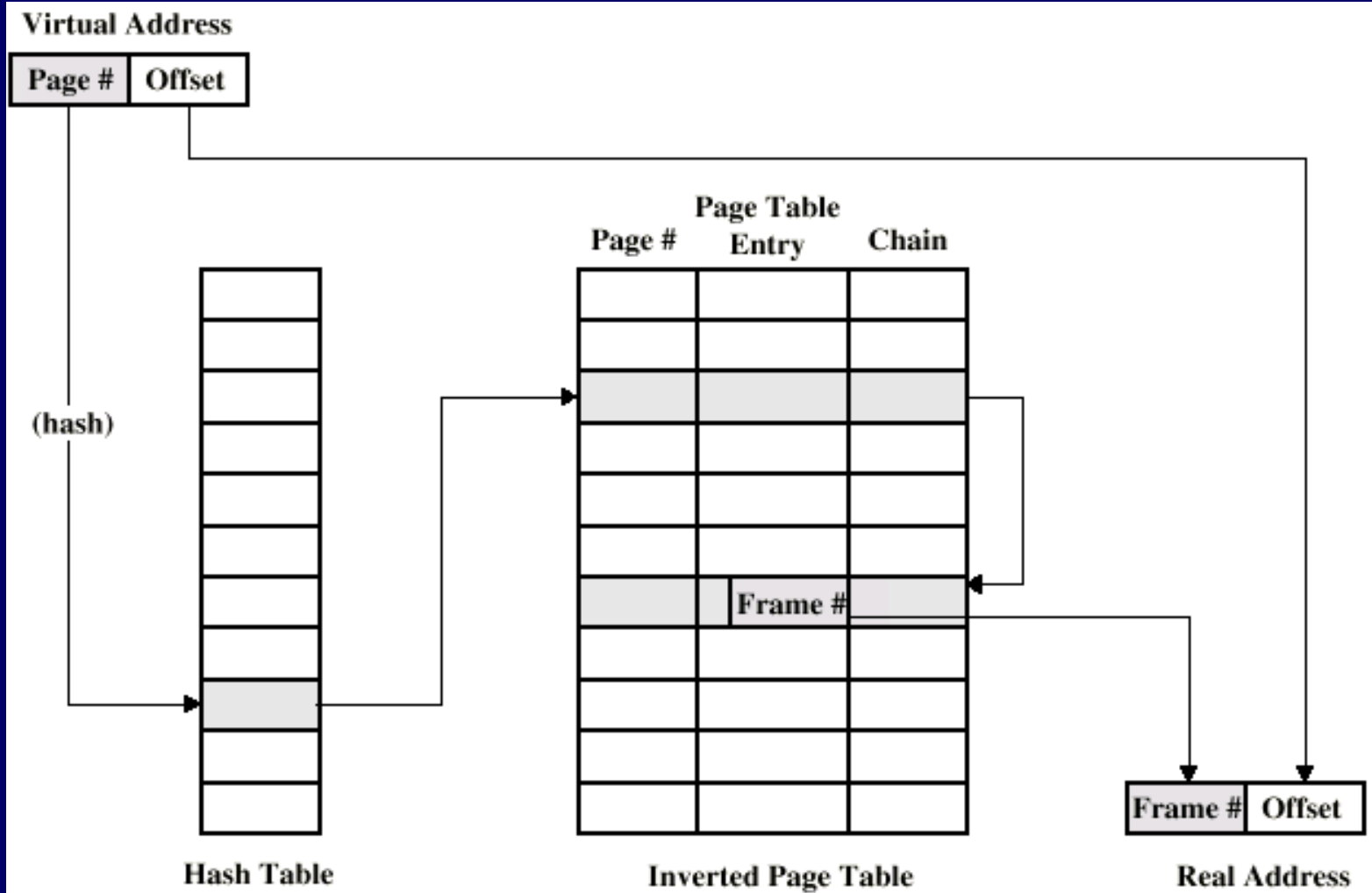
Page Table Structure

- In most systems, one page table per process
 - But each process can occupy huge amounts of virtual memory
 - Amount of memory devoted to page tables could be unacceptably high
 - To overcome: store page tables in virtual memory – page tables subject to paging
 - When process runs, at least a part of its page table must be in main memory
- Some processors use 2-level scheme
 - There is a page directory
 - Page directory points to page table

Inverted Page Table

- Alternative to one- or 2-level page tables
- Page number portion of virtual address is mapped into a hash value
- Hash value is a pointer to inverted page table (has page table entries)
- For each real memory page, there is one entry in inverted page table
- A fixed proportion of real memory is required for hash table and IPT regardless of the number of processes
- Chaining is used to manage overflow in hash table because more than one virtual address may map into same hash table entry

Inverted Page Table Structure



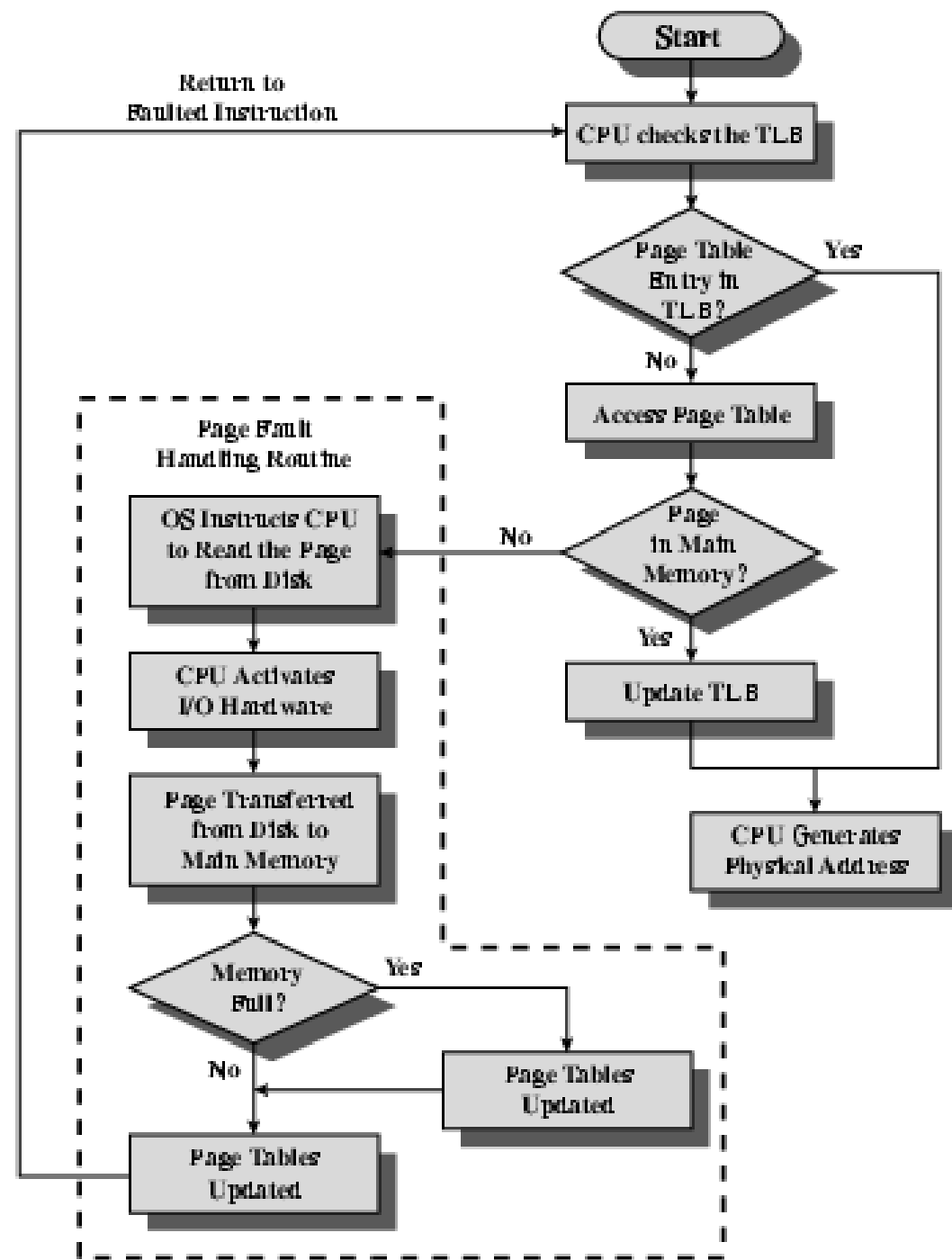
Translation Lookaside Buffer

- Every virtual memory reference causes two physical memory access
 - Fetch page table entry
 - Fetch data
- Increasing memory access time
- To overcome, use special cache for page table
 - TLB

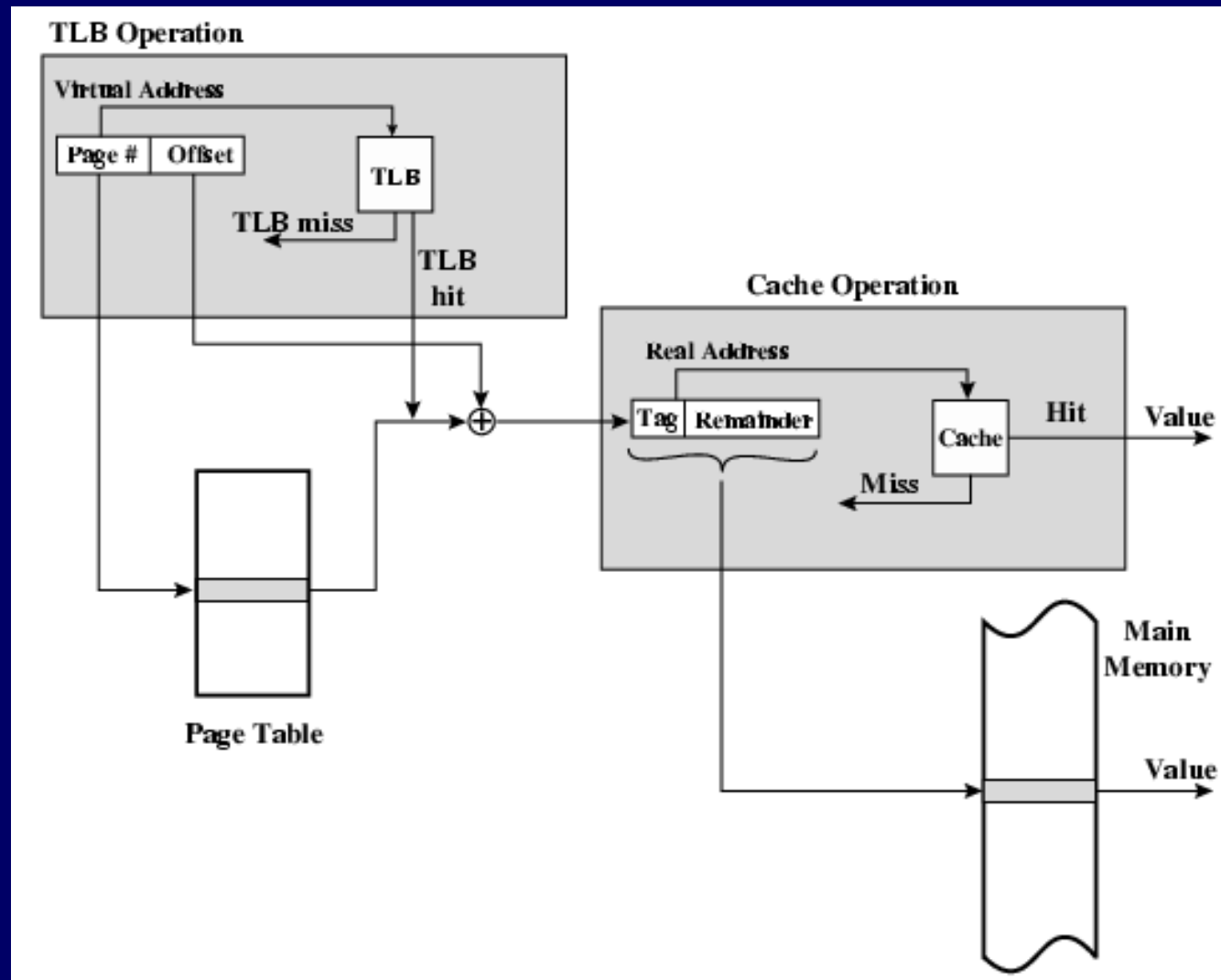
Translation Lookaside Buffer

- Functions same way as memory cache
 - Contains page table entries most recently used
- Virtual memory mechanism interacts with main memory cache mechanism
 - Virtual address (form of page number + offset)
 - TLB consulted to see if matching page table entry is present
 - If present, real address is generated (frame number + offset)
 - If not present, entry accessed from page table
 - With real address (tag + remainder), cache is consulted to see if block containing that word is present
 - If present, word is returned to processor
 - If not present, word is retrieved from main memory

TLB Operation



TLB and Cache Operation



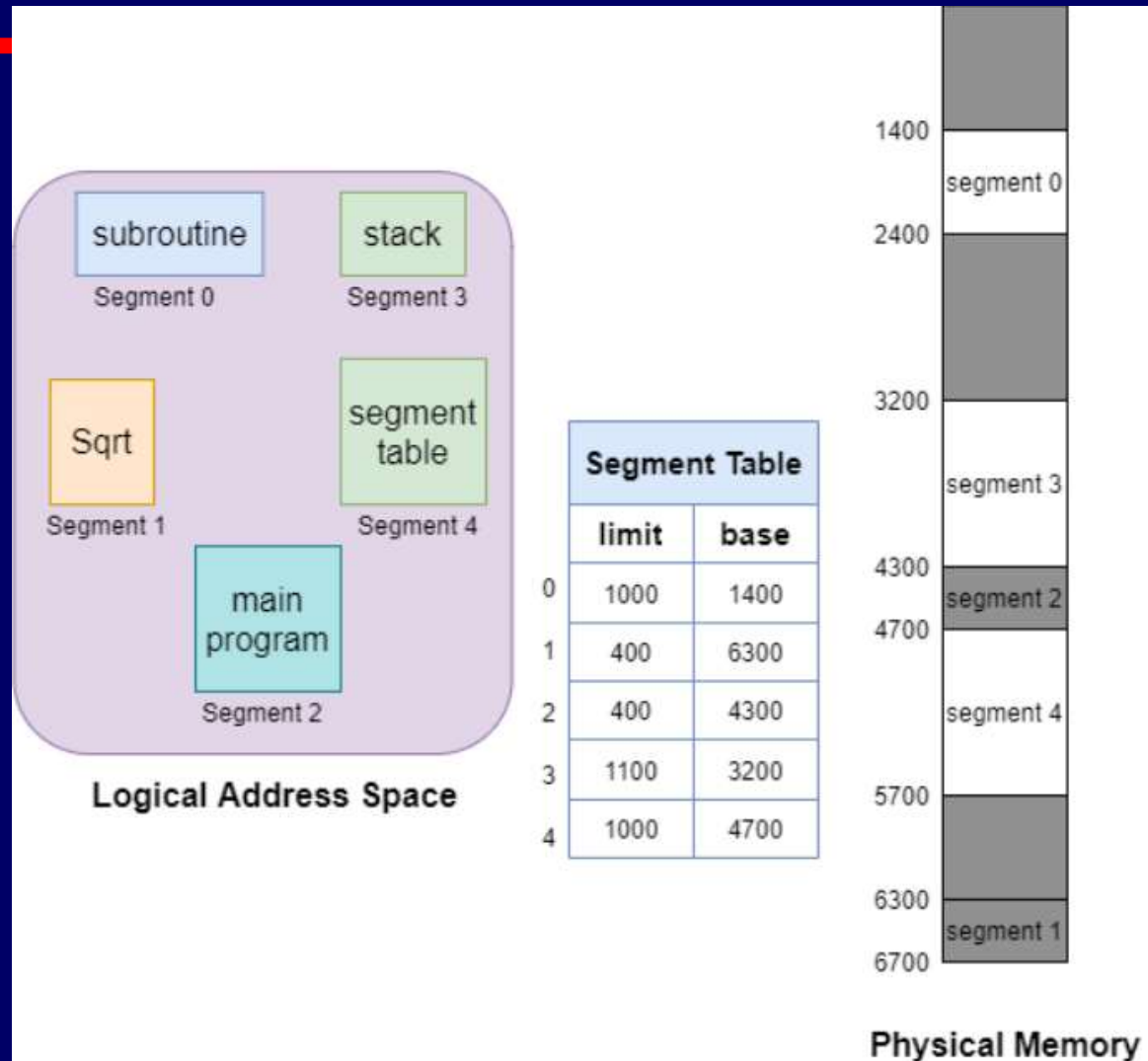
Translation Lookaside Buffer

- So, it is COMPLICATED
 - Virtual address is translated into real address
 - Involves referencing page table
 - In TLB, main memory, or disk
 - Referenced word may be in
 - Cache, main memory, or disk
 - If on disk, word needs to be loaded into main memory, and its block loaded into cache
 - Page table entry for that page must be updated

Segmentation

- Paging is not (usually) visible to the programmer
- Segmentation is visible to the programmer
- Usually different segments allocated to program and data
- May be a number of program and data segments
- Allows programmer to view memory as multiple address spaces or segments
- Segments are variable/dynamic in size
- Programmer (or OS) assigns programs and data to different segments
 - May be a number of program segments (for various types of programs) and a number of data segments
- Each segment may be assigned access and usage rights
- Memory reference consist of segment number + offset

Segmentation



Advantages of Segmentation

- Simplifies handling of growing data structures
 - Segment is expanded or shrunk when needed
- Allows programs to be altered and recompiled independently, without re-linking and re-loading
- Lends itself to sharing among processes
 - Segment can be addressed by other processes
- Lends itself to protection
 - Segment can be assigned privileges
- Some systems combine segmentation with paging

Outline

- Memory Management
 - Swapping
 - Partitioning
 - Paging
- Virtual Memory
 - Demand paging
 - Page table structure
 - Translation Lookaside Buffer
 - Segmentation
- Pentium II/PowerPC Memory Management

Pentium II

- Hardware for segmentation and paging
- Unsegmented unpaged
 - virtual address = physical address
 - Low complexity
 - High performance
- Unsegmented paged
 - Memory viewed as paged linear address space (directory – table – offset)
 - Protection and management via paging
 - Berkeley UNIX

Pentium II

- Segmented unpaged
 - Collection of local address spaces
 - Protection to single byte level
 - Translation table needed is on chip when segment is in memory
- Segmented paged
 - Segmentation used to define logical memory partitions subject to access control
 - Paging manages allocation of memory within partitions
 - Unix System V

Pentium II Segmentation

- Each virtual address is 16-bit segment and 32-bit offset
- 2 bits of segment are protection mechanism
- 14 bits specify segment
- Unsegmented virtual memory $2^{32} = 4\text{Gbytes}$
- Segmented $2^{46} = 64\text{ terabytes}$
 - Can be larger – depends on which process is active
 - Half (8K segments of 4Gbytes) is global
 - Half is local and distinct for each process

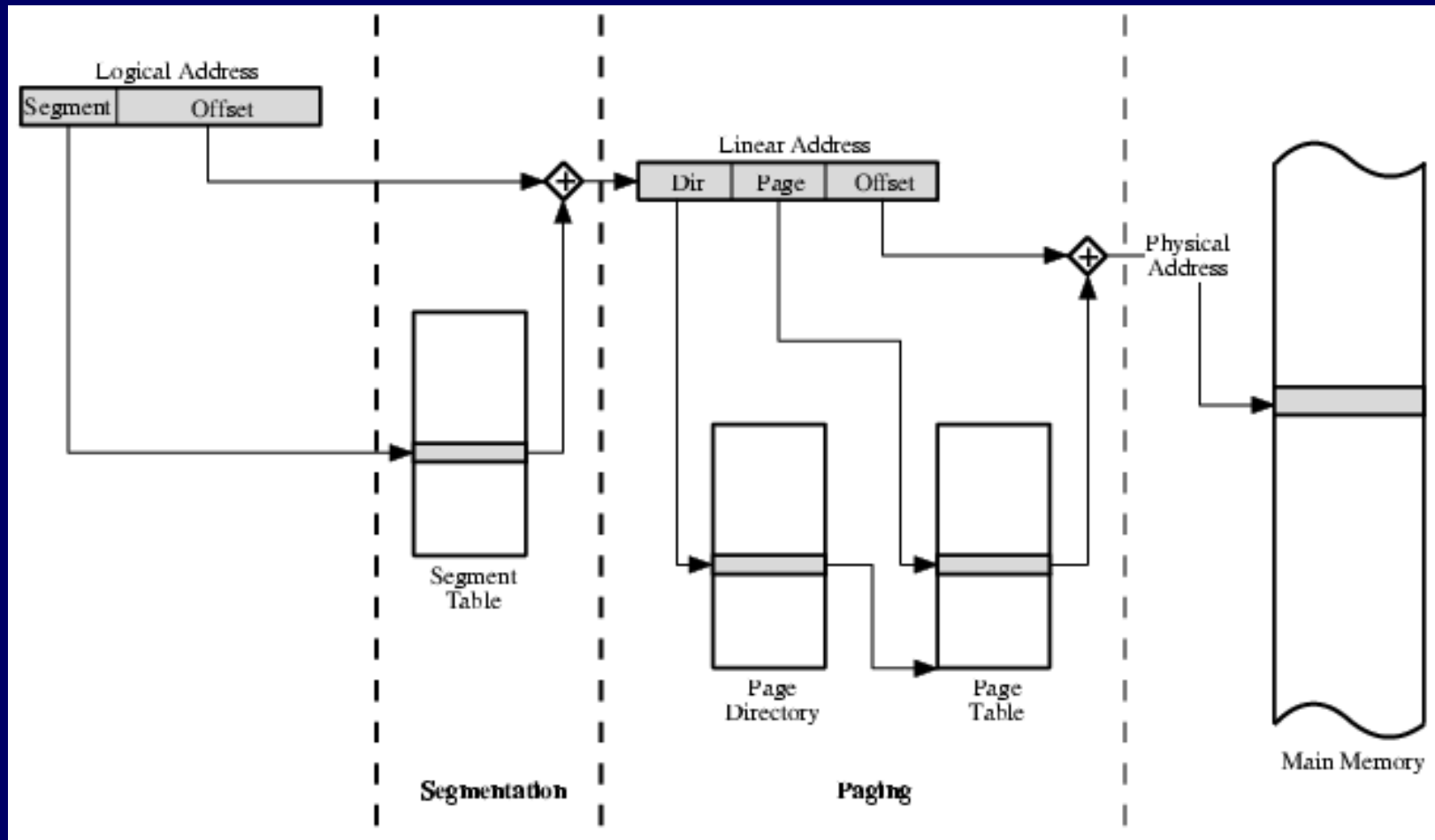
Pentium II Protection

- Protection bits give 4 levels of privilege
 - 0 most protected, 3 least
 - Use of levels software dependent
 - Usually level 3 for applications, level 1 for O/S and level 0 for kernel (level 2 not used)
 - Level 2 may be used for apps that have internal security e.g. database
 - Some instructions only work in level 0

Pentium II Paging

- Segmentation may be disabled
 - In which case linear address space is used
- Two level page table lookup
 - First, page directory
 - 1024 entries max
 - Splits 4G linear memory into 1024 page groups of 4Mbyte
 - Each page table has 1024 entries corresponding to 4Kbyte pages
 - Can use one page directory for all processes, one per process or mixture
 - Page directory for current process always in memory
 - Use TLB holding 32 page table entries
 - Two page sizes available 4k or 4M

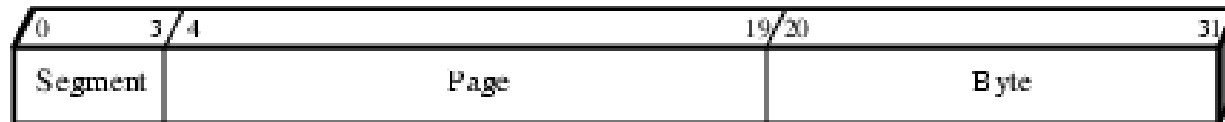
Pentium II Address Translation Mechanism



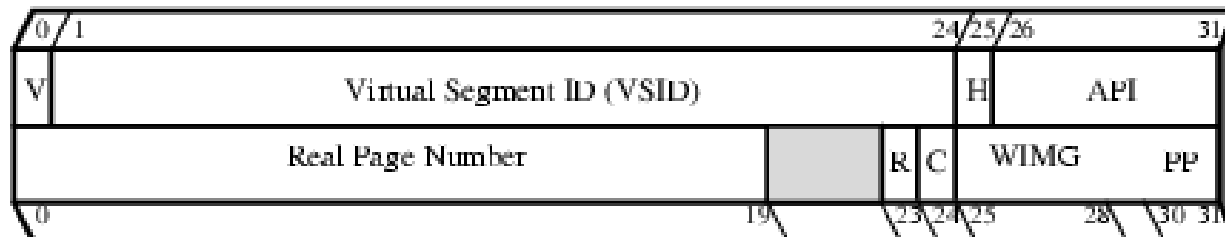
PowerPC Memory Management Hardware

- 32 bit: paging with simple segmentation
 - 64 bit: paging with more powerful segmentation
- Or, 32 & 64 bit: block address translation
 - Map 4 large blocks of instructions & 4 of memory to bypass paging
 - e.g. OS tables or graphics frame buffers
- 32 bit effective address
 - 12 bit byte selector
 - =4kbyte pages
 - 16 bit page id
 - 64k pages per segment
 - 4 bits indicate one of 16 segment registers
 - Segment registers under OS control

PowerPC 32-bit Memory Management Formats



(a) Effective address



V = Entry valid bit

H = Hash function identifier

API = Abbreviated page index

R = Referenced bit

C = Changed bit

WIMG = Cache and storage access control bits

PP = Page protection bits

■ = reserved

(b) Page Table Entry



(c) Real address

PowerPC 32-bit Address Translation

